Projektuppgift

DT211G

Projektuppgift

Ali Aldongy

Mittuniversitetet
MID SWEDEN UNIVERSITY

Campus Härnösand Universitetsbacken 1, SE-871 88. **Campus Sundsvall** Holmgatan 10, SE-851 70 Sundsvall.
Campus Östersund Kunskapens väg 8, SE-831 25 Östersund.
Phone: +46 (0)771 97 50 00. Fax: +46 (0)771 97 50 01.

1 Sammanfattning

Detta projekt består av en webbplats som heter CineSearch där användaren kan söka efter och utforska filmer. Webbplatsen hämtar data från två olika API:er, OMDB och TMDB och presenterar information om filmer som titel, beskrivning och språk.
Webbplatsen är byggd med HTML, SCSS och JavaScript, och använder Vite som automatiserad arbetsprocess.
Användaren kan söka efter filmer, klicka på filmkort för att se detaljerad information, samt spara och hantera sina favoritfilmer.

2 Innehållsförteckning

1 Sammanfattning iiii
2 Innehållsförteckning iv
3 Introduktion 1
3.1 Bakgrund och problemmotivering 1
3.2 Detaljerad problemformulering 1
4 Teori 2
5 Metod 3
6 Konstruktion 4
7 Resultat 7
8 Slutsatser 8
9 Innehållsförteckning 9
10 Kodexempel 10

3 Introduktion

3.1 Bakgrund och problemmotivering

CineSearch skapades i syfte att göra det enkelt för filmälskare att kunna söka efter filmer och få detaljerad information utan att behöva besöka flera webbplatser. Genom att kombinera data från två API:er, OMDB och TMDB, skapas en mer komplett bild av varje film, då båda dessa API:er kompletterar varandra.
Arbetet bygger på att skapa en funktionell webbplats med följande delar:

- En sökfunktion som hämtar filmdata från två olika API:ER.
- En klick funktion där användaren kan se detaljerad information om en vald film.
- En favoritsida där användaren kan spara och hantera sina favoritfilmer.
- En responsiv design anpassad för både mobil och desktop.
- CSS-animeringar för att göra webbplatsen mer levande.
- Eu automatiserad arbetsprocess med Vite för optimering.

3.2 Detaljerad problemformulering

För att uppnå målet med projektet behövdes följande problem lösas:

- API-integration. Webbplatsen behöver hämta och kombinera data från minst två olika relevanta API:er och presentera informationen för användaren på ett enhetligt sätt.
- Användaren ska kunna söka efter valfri film där resultaten uppdateras dynamiskt utan att sidan laddas om.
- När användaren klickar på ett filmkort, ska en popup visas med mer detaljer om vald film.
- Användaren ska kunna skapara favoritfilmer på en separat sida där datan sparas mellan sessioner.
- Webbplatsen ska se bra ut på både större och mindre skärmar.
- CSS-animeringar ska finnas med för att förbättra användarupplevelsen.

4 Teori

SASS/SCSS (Syntactically Awesome Stylesheets) är en utökning av CSS som erbjuder olika funktioner såsom variabler, mixins och villkorssatser. SCSS liknar vanlig CSS i syntax, vilket gör att den är enkel att lära sig om man redan kan CSS. SCSS-filer behöver kompileras till vanlig CSS innan de kan användas i webbläsare[1].

API (Application Programming Interface) är ett gränssnitt som gör det möjligt för program och tjänster att kommunicera med varandra. I detta projekt används två olika webbAPI:er – OMDB och TMDB – som ger data i JSON-format med HTTP anrop[2].

JSON (JavaScript Object Notation) är ett textbaserat dataformat som används för att dela data mellan en server och klient. JSON kan läsas av både människor och maskiner och är standard för modern webbAPI:er[3].

AJAX (Asynchronous JavaScript XML) är en teknik som gör det möjligt att skicka asynkrona HTTP-anrop utan att sidan behöver laddas om. I detta projekt används AJAX i form av fetch API för att hämta data från de två API:er[4].

Vite är ett byggverktyg baserat på Node.js och används för att optimera webbplatser. Vite erbjuder en snabb utvecklingsserver samt optimering av filer för publicering[5].

localStorage är ett webbAPI som gör det möjligt att lagra data i användarens webbläsare. Datan sparas lokalt och finns kvar även när webbläsarens stängs, vilket i denna projekt har använts för att spara favoritfilmer mellan sessioner[6].

5 Metod

För att lösa de problem som nämndes i problemformuleringen valdes följande metoder:

- API-integration: För att hämta data asynkront från OMDB och TMDB användes Fetch API. Data från de två API kombinerades och presenterades i ett kortformat på sidan.

- Sökfunktion: Det skapades ett sökfält med en sökknapp där användaren kan söka efter filmer. Sökningen triggas med både musklick och genom att trycka på Enter. Containern töms och fylls med nya resultat vid varje sökning.

- Detaljvy: En popup-komponent skapas i HTML och visas endast när användaren klickar på ett filmkort. Data lagras som dataset-attribut på korten och visas direkt när popup:en öppnas.

- Favoritsystem: För att spara favoritfilmer mellan olika sessioner användes localStorage. Under varje kort finns en stjärnknapp som låter användaren lägga till eller ta bort filmer från favoriter och en separat favoritsida läser sparad data.

- Responsiv design: Media queries används tillsammans med CSS Flexbox och Grid för att anpassa layouten för olika skärmstorlekar.

- Animeringar: CSS animeringar med keyframes användes för att skapa en flytande hero, pulserade knappar och animerade listpunkter på startsidan, samt en toast-notis med fade animering.

- Automatiserad arbetsprocess: Vite användes som byggverktyg för att optimera webbplatsen. GitHub och GitHub desktop användes för versionshantering med en development branch för utveckling och main branch för publicering.

6 Konstruktion

Webbplatsen består av fyra HTML-sidor. Index.html, utforska.html, favoriter.html och om-oss.html. Alla sidor har samma header med navigering och footer. På utforska sidan finns ett söktfält, en container för filmkorten och en popup-komponent som är dold som standard och visas med JavaScript. Popup:en innehåller element för titel, poster, beskrivning, betyg, antal röster och språk som fylls i med JavaScript när användaren klickar på ett filmkort. På favoritsidan finns bara en container som fylls i dynamiskt med JavaScript vid sidladdning baserat på vad som sparats i localStorage.

För att koppla samman datan mellan OMDB[7] och TMDB[8] användes asynkrona Fetch-anrop med async/await. OMDB returnerar bara grundläggande information, därför krävdes det extra anrop till ett annat API för att hämta fler data som språk, betyg och beskrivning av filmen. TMDB returnerar mer detaljer direkt. Som Figur 1 visar kombineras data från både API:erna och presenteras i ett kortformat.
För varje

Figur 1

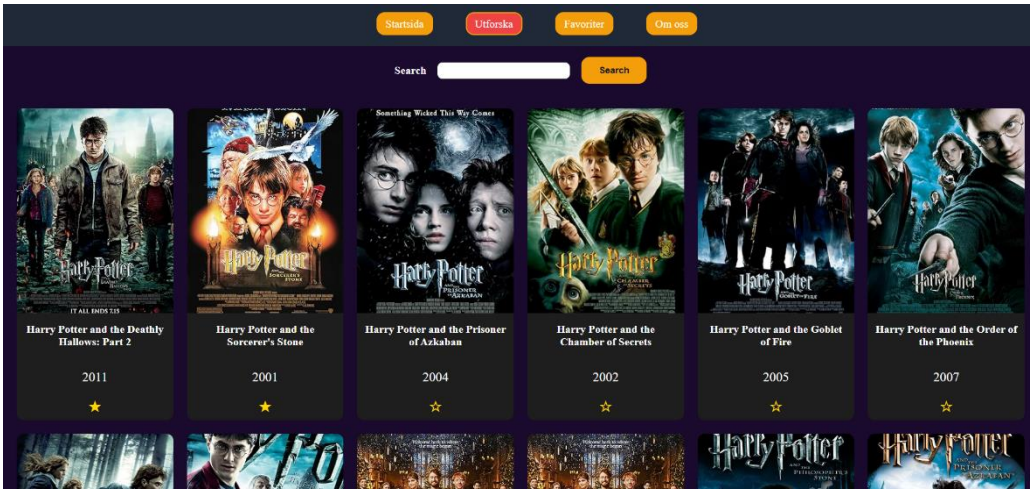
Sökfunktionen implementerades genom att lyssna på både klick på sökknappen och Enter-tangenten. Vid varje sökning töms containern och getData()

funktionen anropas med det nya sökvärdet. encodeURIComponent användes för att hantera specialtecken.

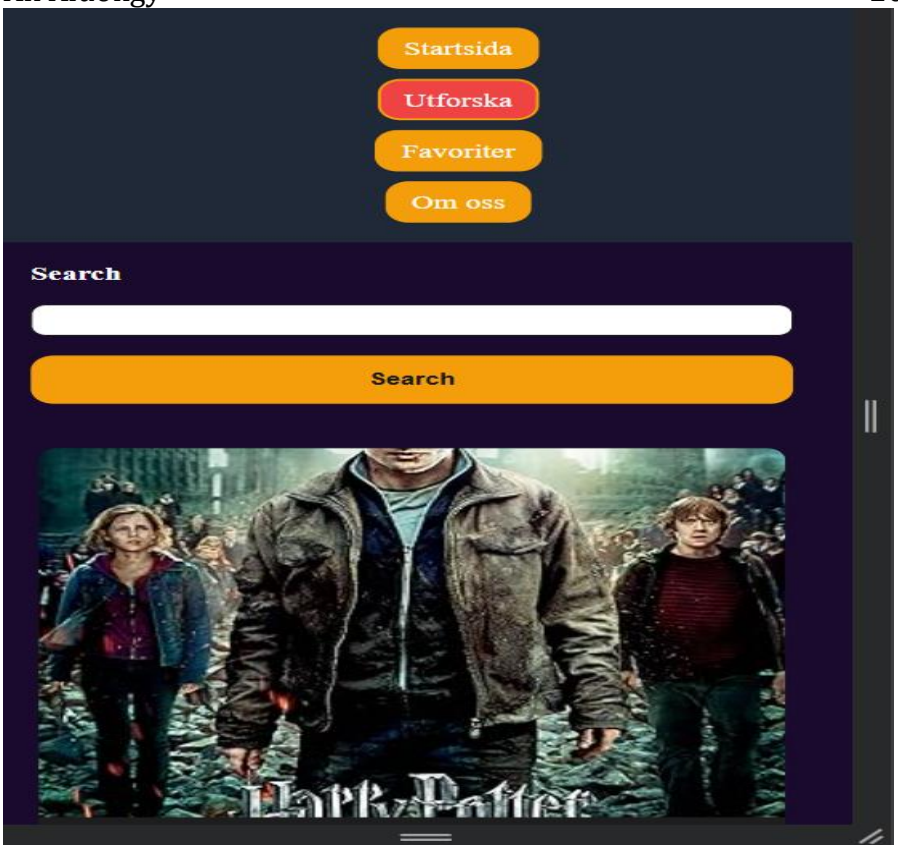
Popup komponenten byggdes i HTML med en overlay bakgrund som stängs vid klick utanför. All filminfo lagras som data-attribut på filmkorten när de skapas och visas när användaren klickar på något filmkort.

Favoritsystemet utvecklades med localStorage där varje film sparas som ett JSON-objekt. När användaren klickar på stjärnknappen som finns under varje filmkort kontrolleras det om filmen redan finns i localStorage. Om den finns tas den bort, annars läggs den till. En toast-notis visas även för att bekräfta det som händer. localStorage läses av på favoritsidan vid sidladdning och korten visas dynamiskt med JavaScript.

Responsiviteten löstes med CSS Grid för filmkortslayouten, där auto-fit och minmax() användes för att anpassa antalet kolumner till skärmbredden. För menyn användes en media quary för maxbredd på 600px där flex-direction ändras till column. Som Figur 3.1 och Figur 3.2 visar fungerar layouten på både mobil och desktop.



Figur 3.1



Figur 3.2

Fyra CSS-animationer skapades med ett tydligt syfte:

- Float: Hero-bilden flyter sakta upp och ner med hjälp av translateY för att skapa ett levande intryck. En box-shadow animeras samtidigt för att förstärka intrycket.

- Pulse: Hero-knappen skalar mellan sin originala storlek och 1.05 större med hjälp av scale, under 2 sekunder och upprepas oändligt.

- Li-animation: Listpunkterna på startsidan animeras med letter-spacing och färgändring för att göra startsidan mer dynamisk.

- FadeInOut: Toast-notisen tonar in med opacity och translateY när den visas, sedan ligger den synlig en liten stund och tonar sedan ut för att ge användaren feedback. Notisen tas också bort med DOM:en efter 2.5 sekunder med setTimeout.

Vite konfigurerades med SCSS-stöd via sass-paket. Projektet versionhanterades med Github Desktop, där development-branchen användes för utvecklingen och main-branchen användes för publicering av webbplats.

7 Resultat

Nedan visas resultatet för samtliga problem från problemformuleringen:

- Integrationen mellan OMDB och TDMB fungerar som planerat. Data hämtas från båda API:erna och presenteras i ett kortformat.

- Sökfunktionen fungerar korrekt och hämtar resultat från båda API:erna samtidigt. Användaren kan söka via knappen eller Enter-tangenten. Vid varje ny sökning, tömts tidiare resultat och ersätts med nya.

- Popup:en fungerar som tänkt. Den visar filmens titel, poster, beskrivning, betyg, antal röster och språk. Den stängs även vid klick på stängknappen eller utanför popup:en.

- Favoritsystemet fungerar som planerat. Det går att lägga till och ta bort filmer från favoriter genom att klicka på stjärnknappen som finns under varje filmkort. Datan sparas i localStorage och finns kvar mellan sessioner. Vid tilläggning eller borttagning av favoritfilm visas korrekt toast-notis.

- Webbplatsen fungerar fint på både mobil och desktop. Filmkortslayouten anpassar sig automatiskt efter skärmbrädd och menyn ändrar riktigt vid 600px och nedåt.

- Samtliga animationer fungerar som tänkt. Hero-bilden flyter, knappen pulsar, listpunkterna animeras och toast-notisen tonar in och ut.

- Vite optimerarr webbplatsen korrekt och Github används för publicering.

8 Slutsatser

Mitt projekt har i stort sett uppnått samtliga mål som nämndes i problemformuleringen. Alla problem har lösts och webbplatsen fungerar som ett användbart verktyg för filmälskare.

Vad som gick bra: Jag skulle säga att integrationen mellan mina två API fungerade smidigt, då de båda kompletterade varandra och erbjöd mycket mer info än vad en enskild tjänst hade kunnat erbjuda. SCSS-strukturen med olika partials gjorde arbetet mycket mer organiserat och lättare att underhålla.

Vad som gick mindre bra: OMDB kräver ett extra HTTP:anrop per film för att hämta detaljerad information. Detta kan leda till att sidan gör flera anrop när en användare söker efter något och kan då upplevas som något långsam. Git-versionshanteringen stötte på ett problem under projektets gång, då en lokal branch(development) blev korrupt, vilket i sin tur krävde felsökning och tog lite extra tid.

Vad som hade kunnat göras annorlunda: Just nu visas bara ett begränsat antal filmer vid sökning. Jag hade kunnat implementera paginering för sökresultaten för att lösa detta. Det hade även varit intressant att använda fler API:er så att fler detaljer läggs till i popup:en, bland annat skådespelare och regissör. Även ge användaren möjlighet att söka efter information om varje film.

Källförteckning

[1] Upwork, ”What is SCSS? A Beginner’s Guide for Developers”,
https://www.upwork.com/resources/what-is-scss
Publicerad 2025-10-13. Hämtad 2026-03-16

[2] Aws, “What is an Api (Application Programming Interface)?”,
https://aws.amazon.com/what-is/api/
Hämtad 2026-03-16

[3] Json, “Introducing JSON”,
https://www.json.org/json-en.html
Hämtad 2026-03-16

[4] W3schools, “AJAX Introduction”,
https://www.w3schools.com/xml/ajax_intro.asp
Hämtad 2026-03-16

[5] Medium, “Easy Explanation about Vite”,
https://medium.com/@gmmicky1026/easy-explanation-about-vite-8a6493731fc4
Publicerad 2024-04-08 Hämtad 2026-03-16

[6] Developer.Mozilla, “Window: localStorage property”,
https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage
Hämtad 2026-03-16

[7] Omdbapi, “The Open Movie Database”,
https://www.omdbapi.com/
Hämtad 2026-03-16

[8] Developer.themoviedb, “Get started with the basics of TMDB API”,
https://developer.themoviedb.org/docs/getting-started
Hämtad 2026-03-16

Kodexempel

Som kodexempel 1 visar har jag en getData() funktion som tar emot en parameter med Harry Potter som standardvärde. Funktionen göra asynkrona Fetch-anrop till både OMDB och TMDB med den angivna söktermen. Jag använder även encodeURIComponent för att hantera specialtecken. Skulle något anrop misslyckas kastas ett felmeddelande. Datan från båda API sparas som JSON i variablerna data1 och data2 för vidare bearbetning.

```
async function getData(query = "Harry Potter") {
    try {
        const container = document.querySelector(".container");
        let res1 = await fetch(`https://www.omdbapi.com/?apikey=cc03f3bb&s=${encodeURIComponent(query)}`);
        if (!res1.ok) throw new Error("OMDb request failed");
        let data1 = await res1.json();

        let res2 = await fetch(`https://api.themoviedb.org/3/search/movie?api_key=d9b57627d30bdf21b3711be3077d8041&query=$
        {encodeURIComponent(query)}`);
        if (!res2.ok) throw new Error("TMDB request failed");
        let data2 = await res2.json();
```

Kodexempel 1

Som kodexempel 2 visar har jag en createCard funktion där ett div element skapas med klassen movie-card. All filminfo sparas som data-attribut på kortet, vilket gör att datan kan läsas av senare när användaren klickar på kortet för att öppna popup:en.

```
function createCard(title, poster, overview, year, rating, voteCount, language) {
    const card = document.createElement("div");
    card.classList.add("movie-card");
    card.dataset.title = title;
    card.dataset.poster = poster;
    card.dataset.overview = overview || "No overview available";
    card.dataset.year = year;
    card.dataset.rating = rating;
    card.dataset.voteCount = voteCount;
    card.dataset.language = language;
```

Kodexempel 2

Som kodexempel 3 visar har jag med en CSS-keyframe-animering kallad pulse på hero-knappen. Animeringen skalar knappen mellan sin originala storlek och 1.05 gånger större sin storlek under 2 sekunder och upprepas oändligt.

```
.hero-btn {
    animation: pulse 2s ease-in-out infinite;
}

@keyframes pulse {
    0%{
        transform: scale(1);
    }
    50%{
        transform: scale(1.05);
    }
    100%{
        transform: scale(1);
    }
}
```

Kodexempel 3